

第2章 コンパイラの概要

2.1 コンパイラの構造

コンパイラはプログラマーが記述する原始プログラムを**フェーズ** (phase) と呼ばれるいくつかの段階を経て計算機が直接実行可能な 0,1 の系列である機械語で記述された目的プログラムへ変換する。目的プログラムは**バイナリコード**とも呼ばれる。

コンパイラの標準的な構造を図 2.1 に示す。

字句解析 (lexical analysis) 字句解析は、アルファベットの系列としての原始プログラムを入力として、プログラミング言語で定められる単語に切り分ける。プログラミング言語の多くは基本的な記述として、1 バイトの文字コードを用いる。一般的には ASCII コード (ANSI X3.4-1986) を用いる。プログラミング言語ごとに定められた予約語 (keyword)、識別子 (identifier)、定数 (constant)、文字列リテラル (string literal)、区切り記号 (delimiter) などを**トークン**として認識し、原始プログラムをトークン列に変換する。トークンは有限オートマトンによって定義される。

構文解析 (parsing) 構文解析は、字句解析の結果出力をプログラミング言語を定義する文脈自由に従ってプログラムの基本的意味を表現する**解析木** (parse tree) を構成する。解析木は構文木 (syntax tree) ともいう。解析木は、プログラムが構文的に持つ情報を表現する。

中間コード生成 (intermediate code generation) 構文木はプログラミング言語が記述するプログラムの意味を表し、構文木をトラバースすることによってプログラムが記述する操作を抽出することができる。計算機ハードウェアに依存しない制御構造とデータフローを中間コード言語のプログラムで表す。中間コードとして、スタックマシンの命令であるバイトコードや3番地コードが用いられる。中間コードは機械語の目的コードを生成するための前段階であり、**フロントエンド** (front end) と呼ばれる。

コード最適化 (code optimization) 解析木から生成される中間コードはプログラムの持つ意味を直接的に表す。しかし、重複した操作が繰り返し実行されるなどの冗長な操作を含む。計算の意味を保存しながらより効率的なコードを生成することを**コード最適化** (code optimization) という。コード最適化は、計算の意味を保存することが要求される。コード最適化は計算の意味を保たれる範囲でより効率的なコードを発見的に生成する。このため、最適化されたコードが必ずしも最も効率的であることは保証されない。最も効率的なコードを得ることは計算理論の結果から不可能であることが示される。

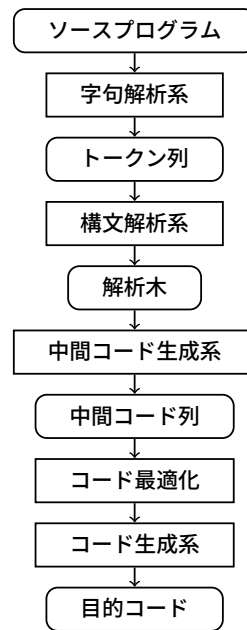


図 2.1: コンパイラの構造

目的コード生成 (object code generation) 目的コード生成では、最適化された中間コード言語のプログラムを機械語のプログラムに変換する。このフェーズでは、中間コードが表す制御フローとデータフローに従って目的コードを実行する計算機の資源を効率的に利用する機械語コードを生成する。計算機ハードウェアでは、演算を実施するためのデータの主記憶からの転送をできるだけ少なくする必要がある。演算のためのレジスタの数はプログラムで利用できるメモリの数は非常に少ないため、プログラムを解析することで効率的にレジスタを利用する機械語プログラムを生成する必要がある。プログラムの操作に対してハードウェアで用意されている効率的な命令を選択する必要がある。

2.1.1 コンパイラにおけるパス

原始プログラムに含まれる表現を一度だけ読んで処理を行う部分を **1パス (pass)** という。コンパイラにおけるいくつかのフェーズは一つのパスにまとめることができ、例えば、字句解析と構文解析は一つのフェーズにまとめることができる。目的コードを原始プログラムを一度だけ読んで生成するコンパイラのことを **1パスコンパイラ (one-pass compiler)** という。実際的なコンパイラは多パスであるものがほとんどである。

2.2 字句解析の概要

字句解析は原始プログラムの文字列をプログラムとして意味のある単語にまとめてトークンの系列として出力する。プログラミング言語で用いられるトークンの種類は以下のようなものがある。

- **予約語 (keyword):** プログラミング言語によって特別な意味を持つ単語。条件分岐を構成する, `if`, `then`, `else` や, 繰返し構造を構成する `while`, `do` などのほかに, 変数に対する組み込みのデータ型を表す `int`, `char` がある。これらの語はプログラミング言語が決まった意味で用いるために特に区別して認識する。
- **識別子 (identifier):** プログラムで用いる変数名, 関数名などに用いる文字系列である。通常, 英数字と一部の記号の系列で構成される。
- **定数 (constant):** 先頭の符号と数字および小数点で構成される数 (例: `-123.4`) や浮動小数点表現 (例: `3.14E1`)。一つの文字 'A' も定数として認識する。
- **文字列リテラル (string literal):** ダブル引用"で囲った文字の並び。"Hello World"のように空白も含めることも可能。
- **区切り子 (delimiter):** プログラムの構文要素を区切る記号。セミコロン `;` は逐次に行う命令の区切りを表す。このほか, 並びの区切りを表すカンマ `,` や括弧 `()`, `{}`, `[]` がある。数式, 論理式において数字や記号を区切る記号 `+`, `-`, `&&`, `=`, `>=` がある。

トークンは, 正規表現を用いて定義することができる。このため, 字句解析を行うためには, トークンを受理する有限オートマトンを構成して, 有限オートマトンの受理系列をトークンとして出力する。

2.3 構文解析の概要

一般にプログラミング言語は文脈自由言語として定義される。このため, プログラミング言語を与える文脈自由文法 G が存在し, 原始プログラムをトークン列に変換した語 w が G の生成する言語に属するかどうか $w \in L(G)$ を判定する。

$w \in L(G)$ のとき, 原始プログラムはプログラミング言語としては誤りがない。このとき, G の開始記号 S から w に至る導出が存在し, 文脈自由言語の特徴から導出の過程は木構造で表すことができる。この木構造は, プログラムの基本的な意味を表現する。例えば, 以下のように文脈自由文法 G を与える。

$$G = \langle \{+, n\}, \{S, E\}, \left\{ \begin{array}{l} S \rightarrow E \\ E \rightarrow E+E \\ E \rightarrow n \end{array} \right\}, S \rangle$$

このとき, $n+n+n \in L(G)$ である。これは, $S \Rightarrow E+E \Rightarrow n+E \Rightarrow n+E+E \Rightarrow n+n+E \Rightarrow n+n+n$ という導出系列が存在するためである。ここで n は自然数に対するトークンであり, 実際には 1, 12, 30 などの数字を字句解析した結果である。

この導出に対する解析木は図 2.2(a) のようになる。

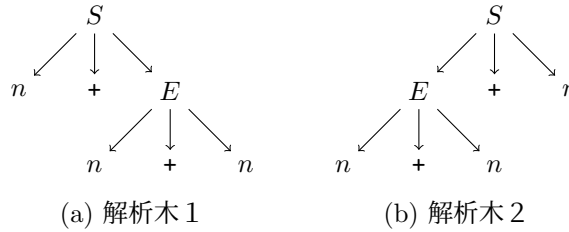


図 2.2: 導出に対する解析木

$n+n+n$ には、別の導出 $S \Rightarrow E + E \Rightarrow E+n \Rightarrow E+E+n \Rightarrow E+n+n \Rightarrow n+n+n$ も存在する。この導出に対応する解析木は (b) のようになる。この導出は $E+E$ において、右の E を n に置き換えてから、左の E を $E + E$ に置き換えている。(a) では $E + E$ の左の E を n に置き換えてから、右の E を $E + E$ に置き換えている。(a) の解釈では、 $+$ を加算とすると、右の 2 つの n を足してから左の n を加算する。(b) の解釈では、左の 2 つの n を足してから右の n を加算する。

$w \notin L(G)$ であれば、 w は文法エラーを含む。通常の言語処理系の構文解析では、単にエラーであることを通知するだけでなく、部分的な構文木を作成して、エラーを起こしている理由をプログラムに提示する。例えば、 $n++n \notin L(G)$ である。最初の $+$ に対して木を構成しようとする $E+E$ の右側の E を $+$ に置き換えることができないため、エラーが生じ、 $+$ の右辺が間違っているといったエラーメッセージが出力される。

コンパイラ構成の観点からプログラミング言語の構文解析に対してはより実用的な要求がある。解析木はプログラムの意味に対応するため、曖昧でない文法が必要になる。曖昧な文法で定義する場合は、どの解析木を導出の結果とするかについてあらかじめ決めておく必要がある。上の文法 G はあいまいで $n+n+n$ に対して図 2.2 の 2 つの解析木が存在する。加算演算は結合的であるので、どちらの木であっても答えは変わらない。しかし、 $+$ を $-$ に置き換えて減算とすると、減算は結合的でないので答えが定まらない。このため、通常は解析木 (b) のように解釈することが決められている。3-2-1 は通常 0 と評価される。解析木 (a) のように解釈すると 2 と評価される。

文脈自由文法 G に対して、 $w \in G$ は決定可能な問題であり、入力の長さに対して $O(n^3)$ で決定することができるアルゴリズムが知られている。コンパイラの構成上、より早い構文解析が必要であり、より高速な構文解析アルゴリズムに合わせてプログラミング言語を定義する文脈自由文法を決定する。決定的に構文解析するための文法のクラスが定義される。

2.4 中間コード生成の概要

中間コードは原始プログラムが持つ基本的な振舞いを抽象的に表現する。中間コードは原始プログラムの手順を表す実行制御フローとデータに対する演算とメモリの更新を表すデータフローを表す。中間言語として、**逆ポーランド記法** (postfix Polish notation あるいは reverse Polish notation)、**抽象構文木** (abstract syntax tree)、**3 番地コード** (three-address code)、バイトコードなどが用いられる。

中間言語が表す振舞いは、プログラムが表す計算手順を標準的な命令単位で並べたものであり、計算機ハードウェアから独立している。中間言語に一旦変換することで、目的プログラムが持つ本

質的な動作意味を単純に表現することができる。このことから異なるアーキテクチャの目的コードは中間コードを変換して生成することができ、移植性が高くなる。

中間言語プログラムでは、変数は**記号表**を作成して実際に割り当てるメモリ番地に変換する。中間言語プログラムの実行環境では、**駆動レコード**とよばれるスタックによって、変数の値を格納する番地を決定する。逆ポーランド記法やバイトコードではデータスタックによってメモリの格納番地を決定する。

このフェーズでは変数の格納番地の決定を行うとともに、データ型など解析木に対してより詳細な制約をチェックする**意味解析**を行う。例えば、 $x+y$ という式に対して、 x と y が同じ型であるかどうか、違う型であっても加算演算が可能かどうかといった**型チェック**を行う。

2.5 コード最適化の概要

コード最適化 (code optimization) は計算の意味を変化させない範囲でより短くて高速な命令系列を導くことである。コード最適化は中間言語プログラムに対して行われるものと目的プログラムに対して行われるものがある。コード最適化は命令を制御が分割したり合流したりしない**基本ブロック**に対して行う**局所最適化** (local optimization) (あるいは**覗き穴最適化** (peep hole optimization) とも呼ばれる)、基本ブロックより広い範囲で行う**大域最適化**がある。その他、数値計算などにおいて最も効果が高い繰り返しループに着目して行う**ループ最適化**がある。一つの手続き範囲 (intra-procedure) で行う最適化、さらに、手続きをまたいで行う (inter-procedure) 最適化がある。最適化は範囲が広がるほど、意味を変えずに効率化することが難しくなる。最適化によって、原始プログラムと目的プログラムの対応をとることが難しくなるので、原始プログラムをデバッグする場合は、最適化を行わない場合が多い。

例えば、図 2.3(a) の中間言語プログラムの断片に対して、同じ計算が繰り返されるているため、(b) のように最適化することができる。(a)、(b) は変数に同じ値を与えると各変数に同じ値を返すので、計算の意味を保存する。

$x=1$	$x=1$
$z=x+y$	$z=1+y$
$w=x+y$	$w=z$
$u=z*4$	$u=z*4$
$v=w*4$	$v=u$
(a) 最適化前	(b) 最適化後

図 2.3: 簡単な最適化の例

大域的な最適化においては**データフロー解析**あるいは**静的単一代入形式**などの方法が用いられる。

2.6 目的コード生成の概要

中間言語コードから実際に実行対象となる計算機ハードウェアが直接実行できる機械語を生成する。目的コード生成では中間言語プログラムの操作をハードウェアの適切な命令に置き換えることと中間コードのメモリをデータ演算を行う際に使用するレジスタに対応させる。ここでは、レジスタの値をできるだけ変更せず、メモリとレジスタの転送を少なくする**レジスタ割り当て**を行う必要がある。

3 番地コードの並び

```
t1=i2-i3
t2=i1+t1
```

を機械語に対応するアセンブリコードにそのまま変換すると以下のようになる。

```
Load   i2
Sub     i3    } t1=i2-i3
Store  t1
Load   i1
Add     t1    } t2=i2+t1
Store  t2
```

ここで、2つめの3番地コードを $t2=t1+i1$ とおきかえてもよい。すると、最初の命令で計算した $t1$ をすぐにメモリに格納する必要はなく、以下のように命令を一つ省略できる。

```
Load   i2
Sub     i3
Store  t1
Add     i1
Store  t2
```

さらに $t1$ がこのあと参照されないことがわかればさらに命令を省略できる。

```
Load   i2
Sub     i3
Add     i1
Store  t2
```

変数の参照と更新に着目すれば、ハードウェアで使用するレジスタの値とメモリとの転送命令 Load, Store を減らして高速化することが可能である。与えられたプログラムに対してレジスタ割り当てを最適にする問題は、有限の組み合わせに対してプログラムを解析することで可解である。しかし、そのアルゴリズムの時間計算量は NP 完全であることが知られているため、レジスタ割り当てによって実行時間を短縮できる効果よりも割当の組み合わせを計算する時間の方が長くなってしまうことが考えられる。このため、実際のコンパイラではより計算量の小さな近似アルゴリズムが用いられる。